

Inputting a list of "high-class C language" code.

 flexonsbd.blogspot.com/2023/05/blog-post.html

Saturday, May 20, 2023

Attempt to create a compiler, Part 1: Inputting a list of "high-class C language" code.

I wanted to try making a compiler, so I entered a list of "high-class C language" code.

I've always dreamed of writing my own compiler, and I've acquired over a dozen books on the subject. However, no matter how many books I read (of course, I haven't read them enough to fully understand them), while I feel like I have a vague understanding of the structure of a compiler (or interpreter), I still don't really understand how to actually construct a program.

My goal is to write my own integer compiler for the 6809, so I'm particularly interested in learning specific code generation techniques. However, most compiler books are very theoretical...

If I were to list some of the books I've read that have left a lasting impression on me,

- A Clear and Concise Introduction to Compiler and Interpreter Development (Haruhiko Hayashi, SoftBank Corporation)

It was easy to understand, but I gave up when I got to the part about using variables in the calculator program.

- Building an Interpreter with Go (by Thorsten Ball, translated by Yoji Shitara, Gijutsu Hyoronsha)

I was impressed by the approach of testing and verifying each step along the way, but I was struggling with the Go language, which I had never used before, so I ran out of steam halfway through.

- Learning to Build a Compiler (by Ikuo Nakata, Ohmsha)

This is, of course, a book by Professor Nakata. I tried to port PL/0, which is covered in this book, to the 6809 using the MicroC compiler on FLEX9, but I was unable to do it.

- Techniques for Creating New Languages Compatible with Z80 CPUs (by Hiroyuki Onuki, MIA Publishing)

Although I have almost no experience with the Z80, this book was the first time I learned about specific methods for code generation. The latter half contains assembly listings for Stellar, a compiler language that runs on the PC-8801 and CP/M-80. (I'd like to try typing in the CP/M-80 version someday. It's over 120 pages long !)

- High-Class C Language (Goro Sueishi, Yu Kobayashi, Gijutsu Hyoronsha)

It's easy to read because it focuses on concrete code generation. It includes the source code for setting the instruction code for a virtual machine, the corresponding compiler, and the interpreter that interprets and executes the generated virtual code.

No matter how many books I read, I couldn't make much progress, so I decided to just start building it myself. However, I've actually started and given up on projects several times before, so I figured starting from scratch would be impossible. Instead, I decided to start by inputting the listings from "High-Class C Language," a book I bought a long time ago and left on my bookshelf, and then replacing the virtual machine part with a 6809.



It turns out that part of the program listing was missing.

I tried inputting the compiler program `mcc` and the interpreter program `mai`.

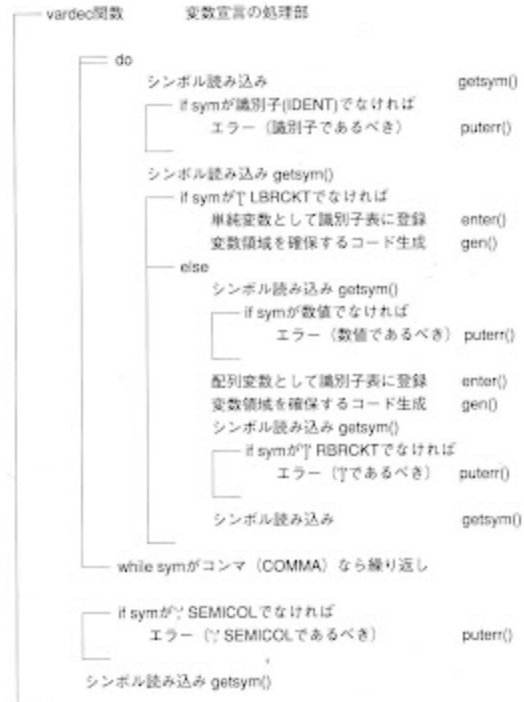
I only realized this after trying it out, but it turns out that the listing for `c_dec.c`, one of the eight files in `mcc`, is missing... (That's a pretty bad typo...)

However, I knew there were three functions within it, and their processing details were written in the main text, and since I had already entered data into the other files, I resigned myself to the fact that I had no choice but to write them myself.

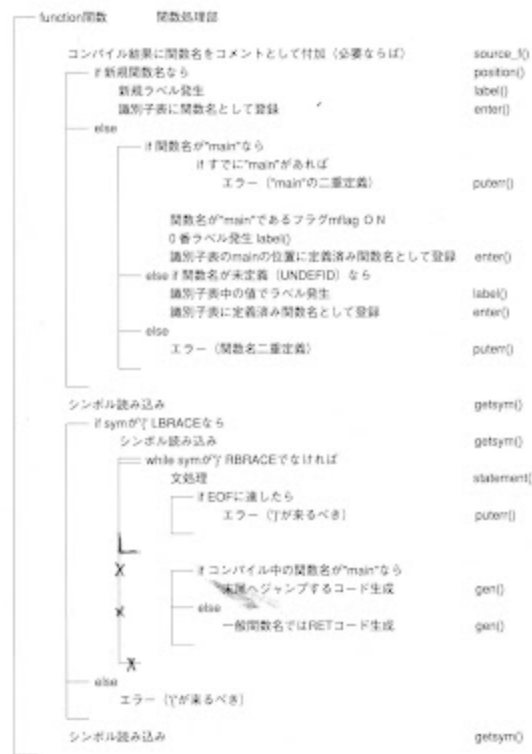
(Incidentally, the copy I own is the first edition from 1993 and has a lot of typographical errors, but since it's 30 years old, I can't find any related information on the publisher's website...)

The three functions you need to create are `vardec()`, which handles variable declarations; `function()`, which processes functions; and `statement()`, which compiles statements.

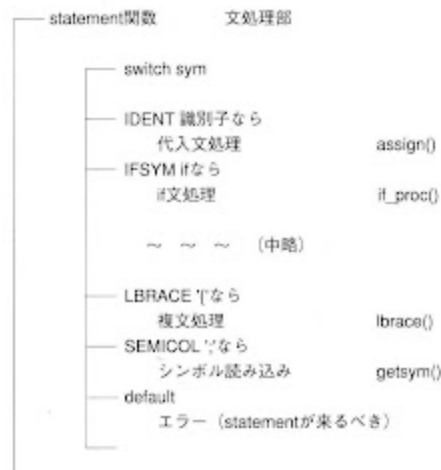
This is an explanation of the specifications.



vardec function specifications



Function specifications



Specifications of the statement function

After struggling for nearly two weeks, I finally managed to get the program into an error-free state, but I feel like I gained a much better understanding of the program's structure during

that process, so I'll consider it a worthwhile experience in the end.

This book is 30 years old, and I doubt anyone is trying to input lists into it now, but I've included the `c_dec.c` file I wrote below. (Adding this to the list in the book should make it work correctly.)

```

1      /*****
2      ****
3      **** c_dec.c Micro C Compiler declaration procedure file <span>
4      </span>****
5      ****
6      *****/
7      #include <stdio.h>
8      #include <stdlib.h>
9      #include <string.h>
10     #include <ctype.h>
11     #include "c_mcc.h"
12     /*****
13     *      vardec()      変数宣言の処理
14     *      戻り値:      なし
15     *      引数:      なし
16     *****/
17     void vardec(void)
18     {
19     do {
20     getsym();
twenty  if (sym != IDENT)
one     puterr("Variable ID expected.");
twenty  getsym();
two     if (sym != LBRCKT) { /* 単純変数 */
twenty  enter(VAR_SMPL);
three   gen(VAR_SMPL, lblcnt++);
twenty  }
four     else { /* 配列変数 */
twenty  getsym();
five    if (isdigit(sym) != 0)
26     puterr("Number expected.");
27     enter(VAR_ARY);
28     gen(VAR_ARY, lblcnt++);
29     getsym();
30     if (sym != RBRCKT)
31     puterr("'']' expected.");
32     getsym();
33     }

```

```

34     } while(sym == COMMA);
35     if (sym != SEMICOL)
36         puterr("' ';' expected.");
37     getsym();
38     }
39     /*****
40     *      function()      関数処理
41     *      戻り値:   なし
42     *      引数:     なし
43     *****/
44     void function(void)
45     {
46         int i, mflag;
47         int j;
48         source_f();
49         if ((i = position()) < 0) {
50             label(lblcnt++, fp);
51             //      enter(FUNCID);
52             enter(UNDEFID);
53         }
54         else {
55             if (strcmp(id, "main") == 0) {      /* if "main" */
56                 j = 0;
57                 while (strcmp(idtbl[j].name, id) != 0 && j <= idcnt) {
58                     j++;
59                 }
60                 if (j < idcnt) {      /* already exist */
61                     if (idtbl[j].kind != UNDEFID) {
62                         puterr("Symbol redeclared.");
63                     }
64                     else {
65                         mflag = ON;
66                         label(0, fp);
67                         //      enter(BLTFUNC);
68                         idtbl[j].kind = BLTFUNC;
69                         idtbl[j].addr = j;
70                     }
71                 }
72             }

```

```

73     else if (idtbl[i].kind == UNDEFID) {
74         label(idtbl[i].addr, fp);
75         //         enter(FUNCID);
76         idtbl[i].kind = FUNCID;
77         //         idtbl[i].addr = lblcnt;
78     }
79     else
80         puterr("Symbol redeclared.");
81     }
82     getsym();
83     if (sym == LBRACE) {
84         //         i = 0;
85         //         while (strcmp(id, idtbl[i].name) != 0 && i < idcnt)
86             //             i++;
87         //         label(idtbl[i].addr, fp);
88         getsym();
89         while (sym != RBRACE) {
90             statement();
91             if (sym == EOFSYM)
92                 puterr("'}' expected.");
93         }
94         //         if (strcmp(id, "main") == 0 && mflag == ON) { /* if "main" */
95             if (mflag == ON) { /* if "main" */
96                 gen(JUMP, 1);
97                 mflag = OFF;
98             } else
99                 gen(RET, DUMY);
100         //         }
101     }
102     else
103         puterr("'{' expected.");
104     getsym();
105 }
106 /*****
107 *     statement()    文処理
108 *         戻り値:    なし
109 *         引数:     なし
110 *****/
111 void statement(void)

```

```

112     {
113     switch (sym) {
114     case IDENT:
115     assign();
116     break;
117     case IFSYM:
118     if_proc();
119     break;
120     case WHILESYM:
121     while_proc();
122     break;
one    case DOSYM:
two    do_while();
three  break;
124     case CALLSYM:
125     call_proc();
126     break;
127     case INPUTSYM:
128     input_proc();
129     break;
130     case PRINTSYM:
131     print_proc();
132     break;
133     case PRTLNSYM:
134     print_proc();
135     gen(NEW_LINE, 0);
136     break;
137     case LBRACE:
138     lbrace();
139     break;
140     case SEMICOL:
141     getsym();
142     break;
143     default:
144     puterr("statement expected.");
145     }
146     }
147
148

```

149
150
151
152
153
154
155
156
157
158
159
160

c_dec.c

It's embarrassing that it took me two weeks to create a program of only about 160 lines, but I was constantly guessing and verifying the behavior of other functions, so I guess it's about right...

The specifications for the resulting mc compiler/assembler language are based on a 2-byte integer type, and the hit & blow example shown below should give you a general idea of how it works.

```
var flag[10], endflag, once_more;
println "*** Hit & Blow ***";
println "Guess 4 digit number (Miunus to abort)";
print "Once more ( YES: 1, NO: other ) ? ";
} while (once_more == 1);
println "See you again!";
while (flag[ans[i]] != 0)
println "Too Big. Re-enter!";
in[1] = (inp % 1000) / 100;
in[2] = (inp % 100) / 10;
if (ans[j] == in[i]) blow = blow + 1;
if (j == 1) if (ans[j] == in[i]) hit = hit + 1;
else println "Hit=", hit, " Blow=", blow;
```

As an example of compiler extensions, this document explains language specification extensions (addition of function arguments, return values, and local variables) and procedures for generating code for a specific CPU. Based on this, if we can replace the part that generates virtual machine code with 6809 instructions, we should be able to create a 6809 compiler, which I would very much like to achieve.

That concludes my introduction to trying out the compiler and interpreter list from the book "High-Class C Language".